



DENA-CORE services for [Client App]

Ficha del Documento

Author	DENA-NEU support office
Document Description	DENA-CORE services for [Client App]

Version control

Versión	Date	Author	Description

TABLE OF CONTENTS

1	CORE SERVICES FOR [CLIENT APP].....	4
2	APP / CLIENT INSTALLMENT INIT	5
2.1	MAIN OBJECTS INVOLVED IN INIT	6
2.1.1	Person	6
2.1.2	Client Installment	6
2.1.3	Client App	6
2.1.4	Config	6
2.2	INIT FUNCTION	7
2.2.1	Client Installment Init Status data	7
2.2.2	Client Installment Startup data	7
2.2.3	How [client app update action] is determined	8
2.2.4	How the [client app status] is determined	8
2.2.5	How the [app init config] is determined	8
2.2.6	Sample messages	10
3	GET INFO ABOUT CHANGES IN DATA (SRMD)	15
3.1	SRMD SENT FROM DENA-APP TO DENA-CORE	16
3.2	SRMD RECEIVED FROM DENA-CORE AS RESPONSE	17
4	RETRIEVE DATA FROM ADMIN.....	19
4.1	HIGH LEVEL VIEW	19
4.2	LOW LEVEL VIEW	1
4.3	HOW DENA-APP KNOWS IF A [DATA ITEM] IS NEW OR UPDATED (HAS CHANGED)	3
5	JAVA API.....	4
5.1	INIT THE [CLIENT INSTALLMENT]	4
5.2	GET INFO ABOUT [CHANGES] IN [DATA] (SRMD)	6
5.3	RETRIEVE [DATA]	8

1 CORE SERVICES FOR [CLIENT APP]

This document describes the **services offered by DENA-CORE for DENA-APP** (*client apps*)

The CORE-Services are:

1. **Client installment INIT**: called by DENA-APP to receive info that determines the UI functionality
2. **SRMD Sync** called by DENA-APP periodically to receive info about [changes] of [data] in the [data origins] (admins)
3. **SRMD Retrieve** called by DENA-APP to get the full [data] from the [data origin] (admin) whenever a [change] of [data] is detected

It's important to understand the DENA architecture basis and DENA configuration before diving in this document

2 APP / CLIENT INSTALLMENT INIT

The [client app] (*mobile/ web*) needs some config data to provide its functionality

- The [org structure] of the [admins] participating in DENA (*needed for example to “paint” org selection UI controls*)
- The [data types] available (*needed for example to “paint” [data type] selection UI controls*)
- The [taxonomies] (*e.g. themes by which the [data types] are categorized*)

... this information should NOT be hard-coded in the [client app], it should be loaded from DENA-CORE each time the [client app] starts

Apart from the previous “static” config about the [org structure], [taxonomies] and [data types] the [client app] needs to show the user **“contextual” information** about which [org structures], [labels] of the [taxonomies] or [data types] he/she has:

- Data available
- New / updated data

For example:

- The user does have [data] in the [label]=family
- The user does have new / updated [data] in the [data type]= administrative records
- The user does have [data] in [admin]=EJGV

... this way, the UI for example can:

- Disable or hide [labels], [data types] or [admins] in which the [user] do NOT have [data]
- Place a badge at [labels], [data types] or [admins] in which the [user] does have new / updated [data]



In the [themes] sub-home depicted the [user] should have some visual hints about:

- Whether *under* a certain [theme] (*a label*) the [user] has (*or has not*) [data] (*the UI can disable or hide that [theme] because if the [user] clicks on the [theme] he/she will NOT find anything*)
- Whether *under* a certain [theme] (*a label*) the [user] has new / updated [data] (*the UI can place a badge in the [theme]*)

The same logic applies to [admins] or [data types]

Finally, the [client app] needs some information about the [app version] so it can **instruct the [user] to update the [app]** or maybe do not allow to use the [app] because there's an ongoing technical work.

2.1 Main Objects involved in init

2.1.1 Person

DENA-CORE has a record of every [person] who has *enrolled* in DENA (*has a DENA account*)

2.1.2 Client Installment

A [person] can use DENA-App (*mobile / web*) in multiple [client devices], for example, a [person] can have many [mobile devices] in which he/she installs the [mobile app] or he/she can open the [web app] in multiple [web browsers]. Each of these [mobile app] *installations* or [web browser] *sessions* is considered a [client installment]

DENA-CORE records the “installments” in which a [person] “installs” DENA-App (*mobile / web*)

For each [installment], DENA-CORE stores:

- The [installment oid] (*generated by the [app]*)
- The [person] owning the [installment]
- The [client device platform] (*MOBILE_ANDROID / MOBILE_IOS / WEB_BROWSER*)
- The [app version]
- The moment in which the [installment] was created

For example, a [person]

- Install [DENA-App mobile] in [mobile device A], [mobile device B]
- Use [DENA-App web] in two [web browser] instances (*e.g. a [chrome] instance and a [firefox] instance*)

... then DENA-CORE will have four [client installments]

2.1.3 Client App

DENA-CORE retains history about [client app] versions (***mobile / web***) versions in terms of:

- The [version number] with the pattern {major}.{minor}.{patch}[-alias] (alias is optional)
- Whether the [version] is a **developer preview**
- The [version] **life range** (start..end) -*if the end bound is null, it's the current version, so just one [version] without upper bound should be registered in the system-*
- The [version] **status**: PERMANENT / TEMPORARY DISABLED or ENABLED
- The target [**client platform**]: MOBILE_ANDROID / MOBILE_IOS and WEB_BROWSER
- **Messages** to be shown to the user after a [version] update

2.1.4 Config

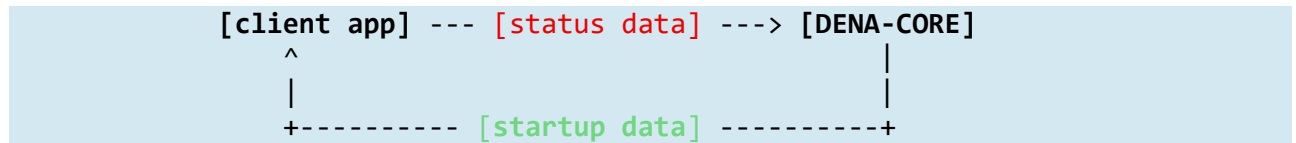
The config needed by DENA-APP to provide it's functionality:

- Taxonomies (*labels*) as [themes]
- Org config: admin groups / admins
- Data Types

2.2 INIT Function

When a [client app] is started and **just after the [person] has identified in the system, the DENA-APP MUST issue a [init] call to DENA-CORE**

The [init] function is straightforward:



As depicted above, the [client app] sends [status data] about the [client installment] to [DENA-CORE] which in turn returns [startup data] in response

2.2.1 Client Installment Init Status data

The [status data] sent by the [client installment] to DENA-CORE just after the [person] has identified in the system contains the following information:

- A [client installment] **unique identifier** (oid)
 - If the [client installment] is a **brand new** [client app] installation (*a new [mobile app] installation or a new [web browser] instance*) the [client app] **must generate a NEW [client installment oid]** and **store it locally for ALL [client installment] life**
 - If the [client installment] is NOT new (*existing installment*), the [installment oid] is **the one stored locally**
- The [app version]: this should be a fixed value at the [app] code
- The [client platform] (*MOBILE_ANDROID, MOBILE_IOS, WEB_BROWSER*): this should be a fixed value at the [app] code
- The **UI language** the [user] has selected while installing the [app]
- The [person] identified

2.2.2 Client Installment Startup data

In **response** to a [init] call, DENA-CORE returns a [client installment startup data] message to the [client app]

- The [client installment oid] > MUST match the one sent by the [client installment] in the [status data]
- **App Version information**
 - The **latest** [app version] available for the [client platform]
 - The [app version] of ALL the [person]’s [client installments]

This info is useful at the [client installment] to inform the user of things that depends on whether he/she has the [client app] installed in another [platform] or not
e.g. - if the [person] is **INITIALIZING** the WEB [client app] he / she can be recommended to install the [mobile app] if the [person] has not installed it yet
- [client app **update action**]: if the [app] (*mobile app*) must / should be **updated** to the **latest version**
- [client **app status**]:
 - **PERMANENT-DISABLED**: the [app] is NOT usable
 - **TEMPORARY-DISABLED**: there’s ongoing maintenance works so the [app] is not usable for a while
 - **ENABLED**: the [app] can be used normally
- **Technical config** about how the [client app] executes the [sync] and [retrieve] operations:
 - The interval of [sync] cycles
 - The number of times the [client app] retries a failed operation
 - The timeout to connect to DENA-CORE
 - The timeout to receive response from DENA-CORE
 - The overall timeout for DENA-CORE operations

- **Messages** to be shown to the [user] after init, *for example*
 - *If the [init] is just after an update or clean installation, DENA-CORE can send a [message] about new features*
 - *New [admins] or [data types] in DENA*
 - *Advice about DENA usage or security advices*
 - *etc*
- **App Init config:** the config about
 - Data Types supported by DENA
 - Organizations: [admin groups] and [admins] in each group
 - Taxonomies: [labels] in which certain DENA objects as [data types] are categorized
- **Errors** during the [init] function

2.2.3 How [client app update action] is determined

The [client app] (*client installment*) sends the [app version] in the [status data]. With that [app version] the DENA-CORE:

- Load the received [app version] config record and if the **[mandatory update flag]** is ON, the [client app] **MUST** be updated
- Compare the received [app version] with the **latest** [app version] available: if the [app version] received is **lower** than the latest available the [client installment] SHOULD be updated

2.2.4 How the [client app status] is determined

The [client app] (*client installment*) sends the [app version] in the [status data]. With this [app version] DENA-CORE loads the [app version] config record that contains [client app status] info that instructs the [client app] to:

- block PERMANENTLY: it's NOT usable anymore
- block TEMPORARILY because of maintenance ongoing works

2.2.5 How the [app init config] is determined

As stated above the **[app init config]** contains information about the [admins], the [data types] and [taxonomies] (*and it's [labels]*) and is used by the [client app] to "paint" the UI

The [app init config] is created from a kind of "**base config**" or a *template* since the [admins], [data types] and [taxonomies] are something that do NOT usually change so they can be CACHED

AppInitConfig

- + (N) AppTaxonomy -----comes from-----> taxonomy (labels)
 - + (N) AppLabel
 - + (N) DataTypes which are categorized with the [label]
- + (N) AppDataType -----comes from-----> Interop Config (data types)
 - + (N) Labels in which the [data type] is categorized
- + (N) Admin Group -----comes from-----> Org Config
 - + (N) Admin
 - + (N) Data Types offered by the [admin]

As depicted above, every object references the [data types] it relates to:

- Labels > the [data types] categorized under the [label]
- Admins > the [data types] offered
- DataTypes (*obvious*)

Starting with this “base config”, DENA-CORE must determine:

- If the [person] has any [data] and new/updated [data] in each of the [admins] and [admin groups] (e.g. under EJGV [admin])
- If the [person] has any [data] and new/updated [data] in each of the [data types] (e.g. under ADMIN-RECORDS [data type])
- If the [person] has any [data] and new/updated [data] in the [taxonomy] and each of the [labels] of the [taxonomy] (e.g. under a certain THEME taxonomy [label] like FAMILIA)

This process is named as **contextualization** of the [app init config] and every object ([org], [data type], [taxonomy]) is also **contextualized to the [person]-specific [data]**

The **contextualization** is done using:

- To guess if the [person] has [data] for a [data type], the SRMD sync [meta-data] sent by the [admins] about [data types] for the [person] is used.
- To guess if the [person] has new or updated [data] for a [data type] the system compares:
 - The last time the [data type] was RETRIEVED for the [person] from the [admin] (no matter the [client installment])
 - The last time any [data] of the [data type] was UPDATED at the [admin] (this info is received from the [admin] in the SRMD sync [meta-data])

If the last time the [person] RETRIEVED the [data type] is BEFORE the last time the [data] was UPDATED at the [admin] it means that the [person] has some **new or updated** [data] pending to be seen.

BEWARE that since the last time the [data type] was RETRIEVED for the [person] from the [admin] not having the [client installment] into account, the [user] will have a consistent behavior across devices: if the [user] “sees” new / updated [data] in a certain [client device] he/she should NOT “see” the [data] as new / updated in another [client device]

2.2.6 Sample messages

2.2.6.1 [status data] sent by the [client installment]

```
{
  "clientInstallmentOid": "342A705D-4BF8-42CF-999C-F4764C9231AC",
  "appVersion": "1.0.0",
  "clientPlatform": "MOBILE_ANDROID",
  "uiLanguage": "SPANISH",
  "person": {
    "id": "00000001R",
    "oid": "748823DA-5271-4E76-9472-ABBAEC73392B"    // not mandatory
  },
  "resourcesUpdatedAt": "2026-08-16T09:54:16.2242396Z"
}
```

2.2.6.2 [client installment startup data] returned by DENA-CORE

```
{
  "clientInstallmentOid" : "342A705D-4BF8-42CF-999C-F4764C9231AC",
  "appVersionData" : {
    "latestAppVersions" : [ {
      "appVersion" : "1.0.0",
      "platform" : "MOBILE_ANDROID",
      "appVersionLifeRange" : "LocalDate:[2026-01-01..)"
    },
    {
      "appVersion" : "1.0.0",
      "platform" : "WEB_BROWSER",
      "appVersionLifeRange" : "LocalDate:[2026-01-01..)"
    } ],
    "clientInstallmentAppVersions" : [ {
      "clientInstallmentOid" : "342A705D-4BF8-42CF-999C-F4764C9231AC",
      "appVersion" : "1.0.0",
      "platform" : "MOBILE_ANDROID",
      "appVersionInstalledAt" : "2026-08-16T09:54:16.394744300Z"
    },
    {
      "clientInstallmentOid" : "532A725D-3BF1-11CA-123C-F5764C0021AC",
      "appVersion" : "1.0.0",
      "platform" : "WEB_BROWSER",
      "appVersionInstalledAt" : "2026-08-11T11:22:11.394744300Z"
    } ]
  },
  "appUpdateAction" : "NO_ACTION",
  "configForSyncAndRetrieve" : {
    "syncAndRetrieveMaxConnectRetries" : 3,
    "syncAndRetrieveConnectTimeOutMillis" : 5000,
    "syncAndRetrieveResponseTimeOutMillis" : 10000,
    "syncAndRetrieveOverallTimeOutMillis" : 60000,
    "secondsBetweenSyncCycles" : 60
  },
  "appInitConfig" : {
    "taxonomies" : [ {
      "oid" : "78AE0290-0F5A-4FFD-B15C-8744F1F702F1"
      "name" : "Temas",
      "newItemSinceFirstClientInstallment" : false,
      "newOrUpdatedData" : true,
      "dataAvailable" : true,
      "labels" : [ {
        "oid" : "A656F2D6-C9F0-4932-A54F-8B39DBBB230C",
        "name" : "Empleo",
        "newItemSinceFirstClientInstallment" : false,
        "dataAvailable" : false,
        "newOrUpdatedData" : false
      } ],
    },
    {
      "oid" : "F6EB16DC-25D4-42BF-BDAC-48FA6030EFA0",
      "name" : "Vivienda",
      "newItemSinceFirstClientInstallment" : false,
      "dataAvailable" : false,
      "newOrUpdatedData" : false,
    },
    {
      "oid" : "E6AF33F6-1CD3-43E7-B35C-11D9F62360DC",
      "name" : "Educación",
      "newItemSinceFirstClientInstallment" : false,
      "dataAvailable" : false,
    }
  ]
}
```

```

    "newOrUpdatedData" : false
  },
  {
    "oid" : "129E7FF4-71E5-4124-8E57-1E04EF99A4EC"
    "name" : "Cultura",
    "dataTypes" : [ {
      "id" : "ADMIN_NOTICE"
    } ],
    "newItemSinceFirstClientInstallment" : false,
    "dataAvailable" : true,
    "newOrUpdatedData" : true,
  },
  {
    "oid" : "B3554B04-1144-4960-BEA0-55A76BC912F7",
    "name" : "Actividades económicas",
    "newItemSinceFirstClientInstallment" : false,
    "dataTypes" : [ {
      "id" : "ADMIN_NOTICE",
      "oid" : "5C7648E3-C804-4C25-A22B-C014DC30C69A"
    } ],
    "dataAvailable" : true,
    "newOrUpdatedData" : true,
  } ]
} ],
"dataTypes" : [ {
  "ref" : {
    "id" : "ADMIN_NOTICE",
    "oid" : "5C7648E3-C804-4C25-A22B-C014DC30C69A"
  },
  "newItemSinceFirstClientInstallment" : false,
  "name" : "Notificación administrativa",
  "dataAvailable" : true,
  "newOrUpdatedData" : true,
  "labelPicks" : [ {
    "oid" : "A72266B5-934D-47A4-820F-83CC9D6F85BD",
    "pickTimeStamp" : 1786874055872,
    "labelOid" : "B3554B04-1144-4960-BEA0-55A76BC912F7",
    "taxonomyOid" : "78AE0290-0F5A-4FFD-B15C-8744F1F702F1",
    "labelRoleInTaxonomy" : "theme"
  } ],
  {
    "oid" : "1D3C7976-7938-49ED-8402-F576298CAEE3",
    "pickTimeStamp" : 1786874055872,
    "labelOid" : "129E7FF4-71E5-4124-8E57-1E04EF99A4EC",
    "taxonomyOid" : "78AE0290-0F5A-4FFD-B15C-8744F1F702F1",
    "labelRoleInTaxonomy" : "theme"
  } ]
} ],
"orgAdminGroups" : [ {
  "ref" : {
    "id" : "GENERAL_ADMIN",
    "oid" : "C340A9A9-46E1-4B0C-92A6-5608593BD1FF"
  },
  "name" : "Administración General de la CAPV",
  "newItemSinceFirstClientInstallment" : false,
  "dataAvailable" : true,
  "newOrUpdatedData" : true,
  "orgAdmins" : [ {
    "ref" : {
      "id" : "EJGV",
      "oid" : "64DCDE00-DC42-47AB-A479-5043B69FA009"
    },
    "name" : "EJGV",
    "newItemSinceFirstClientInstallment" : false,
    "newOrUpdatedData" : true,
    "dataAvailable" : true,
    "dataTypes" : [ {
      "id" : "ADMIN_NOTICE",
      "oid" : "5C7648E3-C804-4C25-A22B-C014DC30C69A"
    } ]
  } ]
},
{
  "ref" : {
    "id" : "LANBIDE",
    "oid" : "3C7D31A6-7675-446E-B5F7-1707446B4DB6"
  },
  "name" : "Lanbide",
  "newItemSinceFirstClientInstallment" : false,
  "newOrUpdatedData" : true,
  "dataAvailable" : true,
  "dataTypes" : [ {
    "id" : "ADMIN_NOTICE",

```

```

        "oid" : "5C7648E3-C804-4C25-A22B-C014DC30C69A"
      } ]
    } ]
  } ],
  "lastUpdatedAt" : "2026-08-16T09:54:16.394744300Z"
}
}

```

These are the parts of the received [client installment init]:

- The [client installment oid] sent by DENA-APP
 - It's a NEW app installation, this oid is generated by DENA-APP, stored locally and sent in future inits
 - It's NOT a new app installation, it's the locally stored oid generated in the first init after the installation

```
"clientInstallmentOid" : "342A705D-4BF8-42CF-999C-F4764C9231AC"
```

- The [app version] data composed by two parts:
 - The **latest** [app version] available in the system for the [client platform]; it does NOT have to match the DENA-APP version sent by the client if it has an older installation

```

"latestAppVersions" : [ {
  "appVersion" : "1.0.0",
  "platform" : "MOBILE_ANDROID",
  "appVersionLifeRange" : "LocalDate:[2026-01-01..)"
},
{
  "appVersion" : "1.0.0",
  "platform" : "WEB_BROWSER",
  "appVersionLifeRange" : "LocalDate:[2026-01-01..)"
} ]

```

- Info about ALL the [client installments] for the person (this [client installment] and others) This info can be useful for example to advice the [user] using the WEB-BROWSER app that he/she can install the MOBILE-APP version if this version is not in the list.

```

"clientInstallmentAppVersions" : [ {
  "clientInstallmentOid" : "342A705D-4BF8-42CF-999C-F4764C9231AC",
  "appVersion" : "1.0.0",
  "platform" : "MOBILE_ANDROID",
  "appVersionInstalledAt" : "2026-08-16T09:54:16.394744300Z"
},
{
  "clientInstallmentOid" : "532A725D-3BF1-11CA-123C-F5764C0021AC",
  "appVersion" : "1.0.0",
  "platform" : "WEB_BROWSER",
  "appVersionInstalledAt" : "2026-08-11T11:22:11.394744300Z"
} ]

```

- Whether the [user] have to UPDATE the [app] or not:

```
"appUpdateAction" : "NO_ACTION"
```

- Config about how the [client device] should handle the [sync] and [retrieve]:
 - Number of connection retries in case of error
 - Timeouts
 - The interval within SYNC cycles

```

"configForSyncAndRetrieve" : {
  "syncAndRetrieveMaxConnectRetries" : 3,
  "syncAndRetrieveConnectTimeOutMillis" : 5000,
  "syncAndRetrieveResponseTimeOutMillis" : 10000,
  "syncAndRetrieveOverallTimeOutMillis" : 60000,
  "secondsBetweenSyncCycles" : 60
}

```

- The [app init] config that contains information about:
 - The [taxonomies] (labels), [data types] and [org structure] (*admin groups and admins*)
 - Where in the [taxonomies] (labels), [data types] and [org structure] the [person] have:
 - Any [data] (*already seen or not*)
 - New or updated [data] (*the one the [user] have not already seen*)

```
"appInitConfig" : {
  "taxonomies" : [ {
    "oid" : "78AE0290-0F5A-4FFD-B15C-8744F1F702F1"
    "name" : "Temas",
    "newItemSinceFirstClientInstallment" : false,
    "newOrUpdatedData" : true,
    "dataAvailable" : true,
    "labels" : [ {
      "oid" : "A656F2D6-C9F0-4932-A54F-8B39DBBB230C",
      "name" : "Empleo",
      "newItemSinceFirstClientInstallment" : false,
      "dataAvailable" : false,
      "newOrUpdatedData" : false
    },
    {
      "oid" : "F6EB16DC-25D4-42BF-BDAC-48FA6030EFA0",
      "name" : "Vivienda",
      "newItemSinceFirstClientInstallment" : false,
      "dataAvailable" : false,
      "newOrUpdatedData" : false,
    },
    {
      "oid" : "E6AF33F6-1CD3-43E7-B35C-11D9F62360DC",
      "name" : "Educación",
      "newItemSinceFirstClientInstallment" : false,
      "dataAvailable" : false,
      "newOrUpdatedData" : false
    },
    {
      "oid" : "129E7FF4-71E5-4124-8E57-1E04EF99A4EC"
      "name" : "Cultura",
      "dataTypes" : [ {
        "id" : "ADMIN_NOTICE"
      } ],
      "newItemSinceFirstClientInstallment" : false,
      "dataAvailable" : true,
      "newOrUpdatedData" : true,
    },
    {
      "oid" : "B3554B04-1144-4960-BEA0-55A76BC912F7",
      "name" : "Actividades económicas",
      "newItemSinceFirstClientInstallment" : false,
      "dataTypes" : [ {
        "id" : "ADMIN_NOTICE",
        "oid" : "5C7648E3-C804-4C25-A22B-C014DC30C69A"
      } ],
      "dataAvailable" : true,
      "newOrUpdatedData" : true,
    }
  ] },
  "dataTypes" : [ {
    "ref" : {
      "id" : "ADMIN_NOTICE",
      "oid" : "5C7648E3-C804-4C25-A22B-C014DC30C69A"
    },
    "newItemSinceFirstClientInstallment" : false,
    "name" : "Notificación administrativa",
    "dataAvailable" : true,
    "newOrUpdatedData" : true,
    "labelPicks" : [ {
      "oid" : "A72266B5-934D-47A4-820F-83CC9D6F85BD",
      "pickTimeStamp" : 1786874055872,
      "labelOid" : "B3554B04-1144-4960-BEA0-55A76BC912F7",
      "taxonomyOid" : "78AE0290-0F5A-4FFD-B15C-8744F1F702F1",
      "labelRoleInTaxonomy" : "theme"
    },
    {
      "oid" : "1D3C7976-7938-49ED-8402-F576298CAEE3",
      "pickTimeStamp" : 1786874055872,
      "labelOid" : "129E7FF4-71E5-4124-8E57-1E04EF99A4EC",
      "taxonomyOid" : "78AE0290-0F5A-4FFD-B15C-8744F1F702F1",
      "labelRoleInTaxonomy" : "theme"
    }
  ] },
  "orgAdminGroups" : [ {
    "ref" : {
      "id" : "GENERAL_ADMIN",
      "oid" : "C340A9A9-46E1-4B0C-92A6-5608593BD1FF"
    },
    "name" : "Administración General de la CAPV",
    "newItemSinceFirstClientInstallment" : false,
    "dataAvailable" : true,
  } ]
}
```

```
"newOrUpdatedData" : true,
"orgAdmins" : [ {
  "ref" : {
    "id" : "EJGV",
    "oid" : "64DCDE00-DC42-47AB-A479-5043B69FA009"
  },
  "name" : "EJGV",
  "newItemSinceFirstClientInstallment" : false,
  "newOrUpdatedData" : true,
  "dataAvailable" : true,
  "dataTypes" : [ {
    "id" : "ADMIN_NOTICE",
    "oid" : "5C7648E3-C804-4C25-A22B-C014DC30C69A"
  } ]
},
{
  "ref" : {
    "id" : "LANBIDE",
    "oid" : "3C7D31A6-7675-446E-B5F7-1707446B4DB6"
  },
  "name" : "Lanbide",
  "newItemSinceFirstClientInstallment" : false,
  "newOrUpdatedData" : true,
  "dataAvailable" : true,
  "dataTypes" : [ {
    "id" : "ADMIN_NOTICE",
    "oid" : "5C7648E3-C804-4C25-A22B-C014DC30C69A"
  } ]
} ]
} ],
"lastUpdatedAt" : "2026-08-16T09:54:16.394744300Z"
}
```

3 GET INFO ABOUT CHANGES IN DATA (SRMD)

Just after the [init], DENA-APP should issue the first **SRMD-SYNC**:

1. DENA-APP sends ALL the SRMD it has locally stored (*returned by DENA-CORE in the previous SRMD-SYNC*)
2. DENA-CORE
 - a) Loads the SRMD for the [person] that have been received from [admins]
 - b) Compares it with the one received from DENA-APP
 - c) Applies the following logic to guess if DENA-APP should issue a RETRIEVE operation for each [data type]

SRMD is like:

```
{person} | {data type} | {admin} | {last update instant}
```

which mean that “some” [data] for [person] of [data type] has been modified (*or is new*) in the given [admin]

(*the concrete [data] modified is NOT known; only that a [change] in the [data] has occurred is known*)

If DENA-APP sends something like:

```
{person} | {data type A} | {admin X} | {T1}
{person} | {data type B} | {admin X} | {T2}
{person} | {data type C} | {admin Y} | {T3}
```

And the SRMD stored in DENA-CORE received from the [admins] is like:

```
{person} | {data type A} | {admin X} | {T1}
{person} | {data type B} | {admin X} | {T2+1min}
{person} | {data type C} | {admin Y} | {T3+5min}
```

Then DENA-CORE can conclude that:

- [data type A] in [admin X] has NOT been modified: T1 received from DENA-APP = T1 at DENA-CORE
- [data type B] in [admin X] has been modified: T2 received from DENA-APP < T2+1min at DENA-CORE
- [data type C] in [admin Y] has been modified: T3 received from DENA-APP < T3+5min at DENA-CORE

... so DENA-CORE can instruct DENA-APP that it has to **refresh (retrieve fresh [data] from the [admin])** for:

- [data type B]
- [data type C]

[data type A] does NOT have to be refreshed since there's no indication of a [change] in the [data origin] (*admin*)

It's important to note that usually SRMD is like:

```
{person} | {admin} | {data type} | {last update instant}
```

... however, there's an architecture where:

- An [admin] has **multiple [data origins]** for a given [data type] (multiple origin systems for the same [data type])
- The [admin] exposes a **dedicated [data provider] for each [data origin]**

In this architecture, the SRMD record have to provide an **additional metadata** field to provide **routing information** to DENA to get to the [data provider]. That metadata field is the **[data origin instance]**

```
[person] | [admin] | [data type] [data origin instance] | [last update instant]
```

So, DENA-APP must store all the previous fields for SRMD entries

3.1 SRMD sent from DENA-APP to DENA-CORE

DENA-APP must send the locally-stored SRMD to DENA-CORE as:

```
{
  "fromClientInstallment" : "37052959-4485-4AE0-B26C-BEADC03D4883",
  "aboutPerson" : "D647136B-C84C-4535-BD8B-2DC323AEC2DF",
  "items" : [
    {
      "atAdmin" : "7846F08B-B909-4C14-8B34-86549DB12595",
      "someDataWasUpdatedAt" : "2026-08-16T19:13:55.6729513Z",
      "ofType" : "2C0540CE-3600-4E63-8847-90392988BA12",
      "fromDataOriginInstance" : "DEFAULT",
      "cacheItemStatus" : "DATA_RETRIEVED_FROM_DATA_ORIGIN",
      "clientLastAttemptToRetrieveDataAt" : "2026-08-15T19:13:55.6749896Z"
    },
    {
      "atAdmin" : "9E467907-9E70-4A84-B65E-D386771CA5A3",
      "someDataWasUpdatedAt" : "2026-08-16T19:13:55.6759741Z",
      "ofType" : "2C0540CE-3600-4E63-8847-90392988BA12",
      "fromDataOriginInstance" : "DEFAULT",
      "cacheItemStatus" : "DATA_RETRIEVED_FROM_DATA_ORIGIN",
      "clientLastAttemptToRetrieveDataAt" : "2026-08-15T19:13:55.6759741Z"
    }
  ]
}
```

As shown above, the [info] sent by DENA-APP to DENA-CORE in essence is:

```
{person} | {data type}+{data origin instance} | {admin} | {last update instant}
| | | | |
| | | | |
| | | | |
+----> The [person] is at the "header" field: aboutPerson since ALL items refers to the SAME [person]
```

Apart from the previous info, DENA-APP also sends other **informative (not mandatory) data** about the [client cache] status:

- cacheItemStatus: if the [data] was **retrieved** (or *not*) from the [data origin] (admin)
- clientLastAttemptToRetrieveDataAt: the last time the [data] was **retrieved** (or *not*) from the [data origin] (admin)

The info sent from DENA-APP is just the SAME info previously received from DENA-CORE in the SRMD-SYNC.

3.2 SRMD received from DENA-CORE as response

The response from the previous message with the DENA-APP locally-stored SRMD contains a more actual copy of the SRMD from DENA-CORE including instructions for DENA-APP in what to do for every [data type]

```
{
  "aboutPerson" : {
    "id" : "00000001R",
    "oid" : "3E1B189A-072F-4BBE-8DCD-FD68E46F66A8"
  },
  "items" : [ {
    "srmdOid" : "99E992AE-E5E9-44A1-9757-D694530B215B",
    "atAdmin" : {
      "id" : "LANBIDE",
      "oid" : "7915BA5D-2F72-4676-835C-54611D725E09"
    },
    "someDataWasUpdatedAt" : "2026-08-16T19:31:59.3076100Z",
    "ofType" : {
      "id" : "ADMIN_NOTICE",
      "oid" : "7BC45D56-5033-4DCC-B4D7-A393A4E55E9E"
    },
    "fromDataOriginInstance": "DEFAULT",
    "postSyncAction" : "RETRIEVE_DATA_FROM_DATA_ORIGIN"
  },
  {
    "srmdOid" : "29F97473-C664-47DF-B9E3-B2E92ABEA3F2",
    "atAdmin" : {
      "id" : "EJGV",
      "oid" : "AD337210-9486-40AD-A107-4F59596F8E29"
    },
    "someDataWasUpdatedAt" : "2026-08-16T19:31:59.3046110Z",
    "ofType" : {
      "id" : "ADMIN_NOTICE",
      "oid" : "7BC45D56-5033-4DCC-B4D7-A393A4E55E9E"
    },
    "fromDataOriginInstance": "DEFAULT",
    "postSyncAction" : "RETRIEVE_DATA_FROM_DATA_ORIGIN"
  } ],
  "erroneousItems" : [ {
    "item" : {
      "atAdmin" : "AD337210-9486-40AD-A107-4F59596F8E29",
      "someDataWasUpdatedAt" : "2026-08-16T19:31:59.1009243Z",
      "ofType" : "7BC45D56-5033-4DCC-B4D7-A393A4E55E9E",
      "fromDataOriginInstance": "DEFAULT",
      "cacheItemStatus" : "DATA_RETRIEVED_FROM_DATA_ORIGIN",
      "clientLastAttemptToRetrieveDataAt" : "2026-08-15T19:31:59.1034294Z"
    },
    "error" : "The item does not exist at the CORE side (there is no SRMD with the given admin/person/data type)"
  } ]
}
```

As shown above, the [info] returned by DENA-CORE to DENA-APP in essence has two sections:

- The items that could be **correctly processed**

```
{
  "srmdOid" : "99E992AE-E5E9-44A1-9757-D694530B215B",
  "atAdmin" : {
    "id" : "LANBIDE",
    "oid" : "7915BA5D-2F72-4676-835C-54611D725E09"
  },
  "someDataWasUpdatedAt" : "2026-08-16T19:31:59.3076100Z",
  "ofType" : {
    "id" : "ADMIN_NOTICE",
    "oid" : "7BC45D56-5033-4DCC-B4D7-A393A4E55E9E"
  },
  "fromDataOriginInstance": "DEFAULT",
  "postSyncAction" : "RETRIEVE_DATA_FROM_DATA_ORIGIN"
}
```

The important info in each item is the [postSyncAction]: what DENA-APP must do for each [data type]

- **RETRIEVE_DATA_FROM_DATA_ORIGIN:** the client MUST issue the **retrieval of the [data type] from the [data origin]** (admin) because the [admin] has sent SRMD to DENA-CORE saying that some [data] has [changed] after DENA-APP issued the last retrieval
- **DO_NOTHING:** the DENA-APP cached [data] (*during a previous retrieval*) has NOT been modified at the [admin] so there's no need to retrieve it again (*DENA-APP can continue using the cached [data]*)
- **DELETE_DATA_FROM_CLIENT_INSTALLMENT:** remove all client cached [data]. This is not a "usual" operation but can be convenient to force all the [client installments] to clear cached [data]

- The items that could **NOT be processed** because the SRMD sent by DENA-APP has some kind of error (*for example, the corresponding SRMD to the one sent by DENA-APP is NOT present in DENA-CORE*)

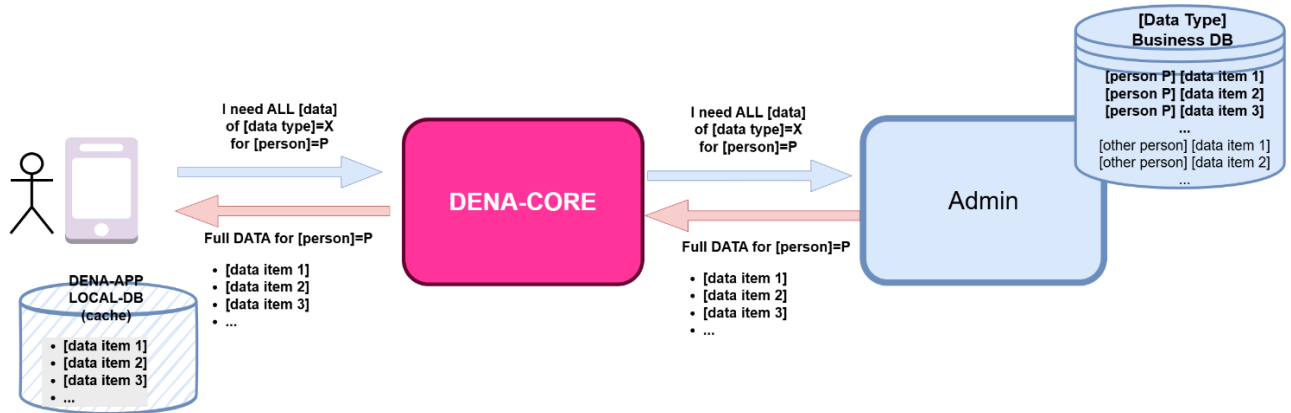
```
{
  {
    "atAdmin" : "AD337210-9486-40AD-A107-4F59596F8E29",
    "someDataWasUpdatedAt" : "2026-08-16T19:31:59.1009243Z",
    "ofType" : "7BC45D56-5033-4DCC-B4D7-A393A4E55E9E",
    "fromDataOriginInstance": "DEFAULT",
    "cacheItemStatus" : "DATA_RETRIEVED_FROM_DATA_ORIGIN",
    "clientLastAttemptToRetrieveDataAt" : "2026-08-15T19:31:59.1034294Z"
  },
  "error" : "The item does not exist at the CORE side (there is no SRMD with the given admin/person/data type)"
}
```

4 RETRIEVE DATA FROM ADMIN

4.1 High level view

[data] **retrieval** is **ALWAYS** initiated by **DENA-APP** (*client installment*) and goes through DENA-CORE directly to the [data origin] at the [admin] who in response returns all the [data]

The high-level schema is like (*read it from left to right*):



As shown, DENA-CORE acts as an intermediary / PROXY for DENA-APP to retrieve the full [data] from DENA-APP

This way, **DENA-APP is decoupled from the [Admins]** and do not need to know anything about them; all the [semantic] complexities are handled by DENA-CORE.

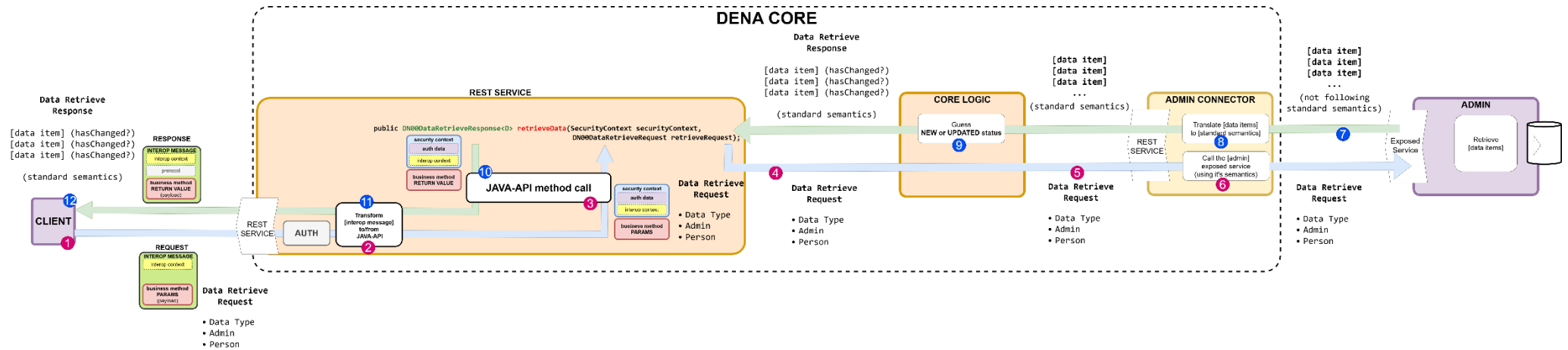
... so, **DENA-APP always talk with DENA-CORE with standard [semantics]:**

- DENA-CORE exposes a REST-service through HTTPS
- DENA-APP identifies itself at DENA-CORE using OAUTH
- The [data] interchanged for every [data type] adjust to a **standard format** (*each [data type] has it's own standard format that models the interchanged info*)

Once the [data] is retrieved, **DENA-APP locally stores (caches) the retrieved [data]** so if there's no indication of a [change] in the [data origin] (admin), DENA-APP can safely "paint" all the UI with the cached [data]

Remember that [admins] periodically sends [change meta-data] (SRMD) that DENA-APP uses to know if it has to retrieve the [data] from the [data origin] (refresh the cached [data])

The following figure resumes all the steps for a [data retrieve] request:



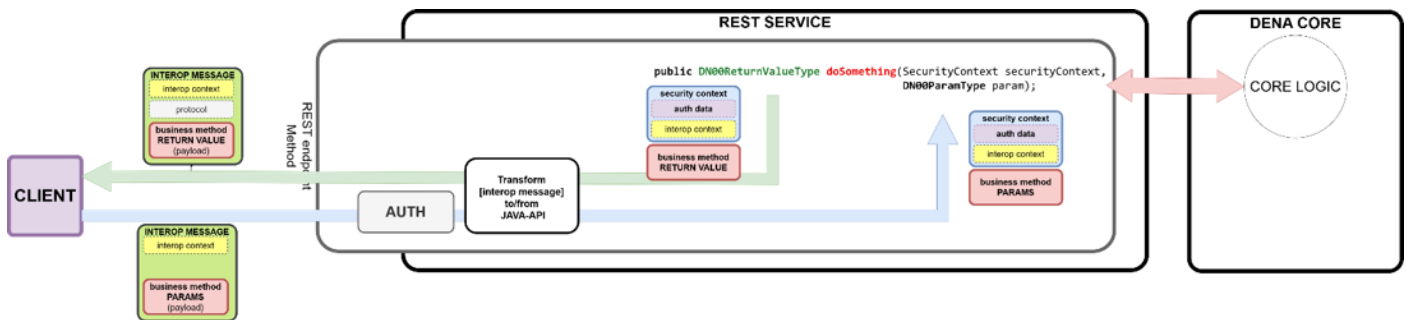
- [1] - The client calls a DENA-CORE REST-Service requesting the [data retrieval] sending an [interop message] that contains the [data type], [admin] and [person] whose data is required
- [2] - The REST-Service authenticates DENA-APP and **translates** the [interop message] to [model objects] used by the JAVA-API
- [3] - The JAVA-API **retrieve data** method is called
- [4] - The JAVA-API calls the CORE LOGIC
- [5] - The CORE LOGIC guess what [admin connector] has to be used and calls the REST-SERVICE exposed (*which talks standard semantics*)
- [6] - The [admin connector] **calls a [data provider] service exposed by the [admin]** that provides the [data tiems]
- [7] - The [data priovder] exposed by the [admin] **retrieves de [data items]** from the [business DB] and returns them to the [admin connector] (*note that the returned data DO NOT HAVE to follow the [standard semantics]*)
- [8] - The [admin connector] -if necessary- **translates the returned [data items]** from an [admin]-custom [semantics] to the [standard semantics]
- [9] - The CORE LOGIC guess the “**has changed**” state of every [data item]
- [10] - The JAVA-API returns the [data items] (*in standard semantics*) with the “**has changed**” status
- [11] - The REST-Service **translates** the JAVA-API returned [data items] to a **response [interop message]**
- [12] - The response [interop message] arrives to the [client]

4.2 Low level view

As previously detailed in the [architecture] doc, DENA-APP exchange [interop messages] with the DENA-CORE REST-Service which is in charge of:

- **Authenticate** the DENA-APP
- **Translate** the received **request** [interop message] into a JAVA-API method **params**
- **Invoke the JAVA-API method** exposed for [data retrieval] which in turn executes the CORE-logic
- **Wrap back the JAVA-API return value** as **response** [interop message] to DENA-APP

```
public <D extends DN00IsDENADDataExchangedObject>  
COREServiceMethodExecResult<DN00DataRetrieveResponse<D>> retrieveData(SecurityContext securityContext,  
DN00DataRetrieveRequest retrieveRequest);
```



[data retrieval] for a [person] from DENA-APP of [data] from fictional [citizen service appointments] is like:

1. The [client installment] (DENA-APP) sends the following [interop message] to DENA-CORE composed of the [interop context] and the [payload]

```
{  
  "context" : {  
    "fromClientInstallment" : "09ECA7C1-7C42-4735-8A48-539675E60D8D",  
    "message" : {  
      "type" : "CLIENT_RETRIEVE_REQ",  
      "correlationId" : "1AB7F413-399D-49F1-AC41-44D651E5799A",  
      "interopRouteData" : [ {  
        "denaComponentId" : "CLIENT_INSTALLMENT",  
        "timestamp" : "2026-08-18T11:28:47.5237003Z"  
      } ]  
    },  
  },  
  "payload" : {  
    "dataType" : {  
      "id" : "citizen_service_appointments",  
      "oid" : "BEDCB4AF-D384-4E05-B74E-A25D7322EF63"  
    },  
    "admin" : {  
      "id" : "admin-id",  
      "oid" : "D4348C40-84B8-4420-9747-193C75CB2875"  
    },  
    "person" : {  
      "id" : "person-id",  
      "oid" : "DAA35E71-5B28-44BF-9DAE-A412E1CEC538"  
    },  
    "clientInstallment" : "ED4576E0-DF47-4D2F-B039-A91228B3F09E"  
  },  
}
```

2. The REST-Service logic just **translates** the [interop message]'s [payload] into the [model object]s used by the JAVA-API method **params**

```
public <D extends DN00IsDENADDataExchangedObject>  
COREServiceMethodExecResult<DN00DataRetrieveResponse<D>> retrieveData(SecurityContext securityContext,  
DN00DataRetrieveRequest retrieveRequest);
```

In this case, the [interop message]'s [payload] is transformed to a DN00DataRetrieveRequest [model object]

3. **The JAVA-API calls the CORE-logic** which uses an [admin connector] to get to the [admin] and ask for the [data]
4. The JAVA-API returns a [model object], in this case a `DN00DataRetrieveResponse<DN00Schedule>` because [citizen service appointments] (schedule items) are being retrieved by DENA-APP.

```
public <D extends DN00IsDENADDataExchangedObject>  
COREServiceMethodExecResult<DN00DataRetrieveResponse<D>> retrieveData(SecurityContext securityContext,  
DN00DataRetrieveRequest retrieveRequest);
```

5. This [model object] (data items) **returned** by the JAVA-API method call is also wrapped back to an [interop message] at the REST-SERVICE:

```
{
  "context" : {
    "fromAdmin" : {
      "id" : "admin-id",
      "oid" : "80996ED2-E7F5-408B-988F-3D9437582D11"
    },
    "message" : {
      "type" : "CLIENT_RETRIEVE_RESP",
      "correlationId" : "1AB7F413-399D-49F1-AC41-44D651E5799A",
      "interopRouteData" : [
        {
          "denaComponentId" : "CLIENT_INSTALLMENT",
          "timestamp" : "2026-08-18T11:28:47.5237003Z"
        },
        {
          "denaComponentId" : "DENA_CORE",
          "timestamp" : "2026-08-18T11:28:47.6217025Z"
        },
        {
          "denaComponentId" : "DENA_ADMIN_CONNECTOR",
          "timestamp" : "2026-08-18T11:28:47.7315121Z"
        },
        {
          "denaComponentId" : "ADMIN",
          "timestamp" : "2026-08-18T11:28:47.7537244Z"
        },
        {
          "denaComponentId" : "DENA_ADMIN_CONNECTOR",
          "timestamp" : "2026-08-18T11:28:47.7722214Z"
        },
        {
          "denaComponentId" : "DENA_CORE",
          "timestamp" : "2026-08-18T11:28:47.8012511Z"
        }
      ]
    }
  },
  "payload" : {
    "requestedNumberOfItems" : 0,
    "itemsPagingContext" : {
      "totalItemsCount" : 200,
      "startPosition" : 0,
    },
    "dataItems" : [
      {
        "data" : {
          "type" : "scheduleItem", // id used to handle polymorphic types (all [data] types implement DN00IsDENADDataExchangedObject
          "id" : "appointment123",
          "lastChangedAt" : "2026-08-19T08:07:56.7423565Z", // very important!!!
          "aboutPerson" : {
            "oid" : "D530141A-1E2A-4800-A118-3FC8D6EFE6D5"
          },
          "year" : "2026",
          "monthOfYear" : "8",
          "dayOfMonth" : "18",
          "hourOfDay" : "13",
          "minuteOfHour" : "2",
          "durationMinutes" : 30,
          "subject" : {
            "ENGLISH" : "an appointment",
            "SPANISH" : "una cita"
          },
          "urls" : [ {
            "id" : "main",
            "lang" : "BASQUE",
            "value" : "https://my-admin.eus/citizen-service-appointments/appointment123?lang=eu"
          },
          {
            "id" : "main",
            "lang" : "SPANISH",
            "value" : "https://my-admin.eus/citizen-service-appointments/appointment123?lang=es"
          } ]
        },
        "isNewOrUpdated" : false
      },
      {
        "data" : {
          "type" : "scheduleItem", // id used to handle polymorphic types (all [data] types implement DN00IsDENADDataExchangedObject
          "id" : "appointment456",
          "lastChangedAt" : "2026-08-19T08:07:56.7423565Z", // very important!!!
          "aboutPerson" : {
            "oid" : "D530141A-1E2A-4800-A118-3FC8D6EFE6D5"
          },
          "year" : "2026",
          "monthOfYear" : "9",
          "dayOfMonth" : "21",
          "hourOfDay" : "10",
          "minuteOfHour" : "20",
          "durationMinutes" : 15,

```

```

    "subject" : {
      "ENGLISH" : "another appointment",
      "SPANISH" : "otra cita"
    },
    "urls" : [ {
      "id" : "main",
      "lang" : "BASQUE",
      "value" : "https://my-admin.eus/citizen-service-appointments/appointment456?lang=eu"
    },
    {
      "id" : "main",
      "lang" : "SPANISH",
      "value" : "https://my-admin.eus/citizen-service-appointments/appointment456?lang=es"
    } ]
  },
  "isNewOrUpdated" : true
}
]
}
}

```

4.3 How DENA-APP knows if a [data item] is NEW or UPDATED (has changed)

DENA-APP needs to know if each [data item] returned by the [admin] (*for example each [schedule item] in the case of the previous sample*) is NEW or has been UPDATED since the last time the [person] used DENA-APP.

With this information DENA-APP can highlight the NEW or UPDATED [data items]

The [admin] does NOT need to send if a [data item] is NEW or has been UPDATED because **DENA-CORE** guess that state while [data items] passes through DENA-CORE during [data retrieval] (*remember that DENA-CORE does not store [data items]*)

The logic that DENA-CORE uses is:

1. Every [data item] returned by the [admin] has a **lastChangedAt** field with the last instant when that [data item] has changed at the [admin]
2. DENA-CORE loads the **last time the [person] issued a [data retrieval]** (*before this one*)
3. If the last time the [person] issued a [data retrieval] is **BEFORE** the [lastChangedAt] instant of the [data item], then the [data item] is NEW or has been UPDATED

NOTE that the {last time the [person] issued a [data retrieval]} is **loaded for the [person] and NOT for the [client installment]**; this way the behaviour is consistent across [client installments]

5 JAVA API

DENA offers a Java API that makes app integration straightforward. The API has two parts:

- **Model:** plain objects that represent the concepts behind
 - Configuration (org, interop, apps, ...)
 - SRMD
 - Person
 - etc
- **API:** the piece that intermediates with DENA-CORE interchanging [model objects]. In other words, the API exposes DENA-CORE functionality by means of method / functions that talk messages modeled against the [model objects]

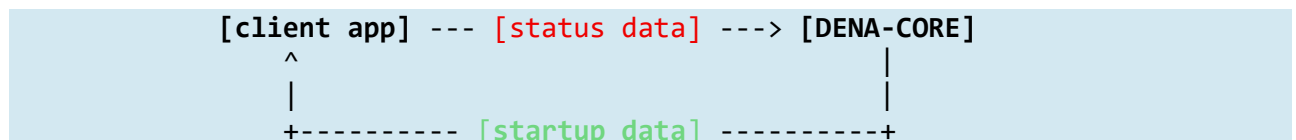
The API for DENA-APP is:

5.1 Init the [client installment]

DENA-APP just after the [user] logs-in MUST issue a [init] with DENA-CORE to:

- Register the [client installment] if it's a NEW installation
- Receive **static config** needed to *paint* the [user interface] (UI)
 - [org config]: [admin groups] and the [admins] in each group
 - [interop config] the [data types] handled by [DENA]
 - Taxonomies: [labels] in which certain DENA objects as [data types] are categorized
- Receive **technical parameters** for DENA-APP:
 - Connection retries and timeouts for [data retrieval] from the [data origins] (admins)
 - Interval between SRMD-SYNC cycles
- Whether the [client installment] must be updated or is usable
- Etc

Recalling from the previously explained, the [init] function is like:



So, the first step for DENA-APP is to *collect* the [status data]

- A [client installment] **unique identifier** (oid)
 - If the [client installment] is a **brand new** [client app] installation (*a new [mobile app] installation or a new [web browser] instance*) the [client app] **must generate a NEW [client installment oid]** and **store it locally for ALL [client installment] life**
 - If the [client installment] is NOT new (*existing installment*), the [installment oid] is **the one stored locally**
- The [app version]: this should be a fixed value at the [app] code
- The [client platform] (*MOBILE_ANDROID, MOBILE_IOS, WEB_BROWSER*): this should be a fixed value at the [app] code
- The **UI language** the [user] has selected while installing the [app]
- The [person] identified

With the previous info DENA-APP creates a `DN00ClientInstallmentStatusData` instance:

```
Instant now = Instant.now();
AppVersion appVersion = AppVersion.from("..."); // should come from an [app] constant
DN00ClientAppPlatform appPlatform = DN00ClientInstallmentStatusData.from("..."); // should come from an [app] constant
DN00ClientInstallmentOID installmentOid = brandNEWInstall ? DN00ClientInstallmentOID.supply() // new installment: generate locally
                                                         : DN00ClientInstallmentOID.forId("..."); // locally stored oid

Language uiLang = // the user-selected lang
DN00PersonRef person = DN00PersonRef.from(DN00PersonID.forId("...")); // the person ID (NIF) from the user that logged in

DN00ClientInstallmentStatusData cliStatus = DN00ClientInstallmentStatusDataBuilder.forClientInstallment(installmentOid)
                                                         .withAppVersion(appVersion, appPlatform)
                                                         .withUILanguage(lang)
                                                         .withPerson(person)
                                                         .build();
```

Now just send this `DN00ClientInstallmentStatusData` to the DENA-CORE [init service] that returns the [client installment STARTUP data]

```
DN00ClientInstallmentInitStartupData cliStartupData1 = _clientInstallmentApi.getForClientInstallmentInit()
                                                         .initClientInstallment(cliStatus);

Log.info("INIT [client installment] startup data:\n" +
        "{}",
        _marshaller.forWriting()
            .toXml(cliStartupData1));
```

5.2 Get info about [changes] in [data] (SRMD)

The first step for a DENA-APP to SYNC meta-data is to **send the locally-stored copy of the SRMD it in a previous SYNC got from DENA-CORE**, so, a collection of [SRMD] items must be created with each DB row.

Supposing the **local** SRMD DB is like:

Column name	Data Type	Details
SRMD_OID	String	The OID assigned by DENA-CORE to the SRMD item
ADMIN_OID	String	Admin OID as received from DENA-CORE
ADMIN_ID	String	Admin ID as received from DENA-CORE
PERSON_OID	String	Person OID as received from DENA-CORE
PERSON_ID	String	Person ID as received from DENA-CORE (NIF)
DATA_TYPE_OID	String	Data Type OID as returned from DENA-CORE
DATA_TYPE_ID	String	Data Type ID as returned from DENA-CORE
LAST_KNOWN_ADMIN_UPDATED_AT	Instant	The instant (date & time) received from DENA-CORE with the last known [change] in the [data]
RETRIEVED_AT	Instant	The instant (date & time) where DENA-APP retrieved the data from the [admin]

Beware that the **ONLY** data that changes from the ones received from DENA-CORE in the SRMD-SYNC is the “RETRIEVED-AT” column that stores the instant (date & time) when DENA-APP retrieved the [data] from the [data origin] (admin)

```
private Collection<DN00SyncMetaDataFromAdminToCOREItem> createSyncMetaDataFromClientInstallmentToCORE(final Collection<LocalDBSRMDRow> localSRMDItems) {
    // for each local SRMD row create a metadata item
    return localSRMDItems.stream()
        .map(localSRMDItem -> {
            DN00SRMDOID srmdOid = DN00SRMDOID.forId(localSRMDItem.get("SRMD_OID")); // not mandatory
            DN00AdminRef admin = DN00AdminRef.from(localSRMDItem.get("ADMIN_OID"), localSRMDItem.get("ADMIN_ID"));
            DN00PersonRef person = DN00PersonRef.from(localSRMDItem.get("PERSON_OID"), localSRMDItem.get("PERSON_ID"));
            DN00DataTypeRef dataType = DN00PersonRef.from(localSRMDItem.get("DATA_TYPE_OID"), localSRMDItem.get("DATA_TYPE_ID"));
            DN00DataOriginInstanceId dataOriginInstanceId = DN00DataOriginInstanceId.DEFAULT;
            Instant lastKnownAdminUpdatedAt = localSRMDItem.get("LAST_KNOWN_ADMIN_UPDATED_AT");
            Instant retrievedAt = localSRMDItem.get("RETRIEVED_AT") != null
                ? localSRMDItem.get("RETRIEVED_AT")
                : null;

            return DN00SyncMetaDataBuilder.syncMetaFromClientInstallmentToCOREItemBuilder()
                .withSRMDoid(srmdOid)
                .atAdmin(admin)
                .someDataWasUpdatedAt(lastKnownAdminUpdatedAt)
                .ofType(dataType)
                .fromDataOriginInstance(dataOriginInstanceId)
                .clientInstallmentRetrievedDataFromDataOriginAt(retrievedAt)
                .build();
        })
        .toList();
}
```

Beware that the **DN00SyncMetaDataFromAdminToCOREItem** do not contain:

- The [person] reference because ALL items refers to the SAME [person], **the one logged-in in DENA-APP**
- The [client installment]: the **OID generated by DENA-APP when installed** an initialized in DENA-CORE

Those values are also stored locally in DENA-APP

With all that, the final message to be sent is built as:

```
DN00ClientInstallmentOID clientInstallmentOid = // ... get [client installment oid] from local-db
DN00PersonOID personOid = // ... get [person oid] from local-db
DN00SyncMetaDataFromClientInstallmentToCORE clientToCORE = DN00SyncMetaDataBuilder.syncMetaFromClientInstallmentToCOREBuilderFor(clientInstallmentOid,
                                                                                                            personOid)
    .withItems(clientToCOREItems);
```

Once the **DN00SyncMetaDataFromClientInstallmentToCORE** object is built containing:

- The [client installment] oid
- The [person] reference
- The collection of SRMD items locally stored at DENA-APP and previously received from DENA-CORE in the last SYNC

... just send the info to DENA-CORE to **receive the UPDATED copy of SRMD** including *instructions* on what to do for each of the [data types]

- **RETRIEVE_DATA_FROM_DATA_ORIGIN**: the client MUST issue the **retrieval of the [data type] from the [data origin]** (admin) because the [admin] has sent SRMD to DENA-CORE saying that some [data] has [changed] after DENA-APP issued the last retrieval
- **DO_NOTHING**: the DENA-APP cached [data] (*during a previous retrieval*) has NOT been modified at the [admin] so there's no need to retrieve it again (*DENA-APP can continue using the cached [data]*)
- **DELETE_DATA_FROM_CLIENT_INSTALLMENT**: remove all client cached [data]. This is not a “usual” operation but can be convenient to force all the [client installments] to clear cached [data]

The code to invoke DENA-CORE is like:

```
DN00SyncMetaDataFromCOREToClientInstallment syncResult = _syncAndRetrieveApi.getForMetaDataSyncForClientInstallment()  
                                                                                      .syncClientInstallmentMetaData(clientToCORE);  
log.info("Received SRMD from DENA-CORE:\n" +  
        "{}",  
        _marshaller.forWriting()  
                      .toJson(syncResult));
```

5.3 Retrieve [data]

The [data] retrieval API is extremely straightforward given **that each [data type]'s [data] has its own [model] (semantics / format)**

Every [data] object implements the java interface *DN00IsDENADDataExchangedObject* so it's quite easy to find the [data] models for every [data type]: just find java types extending *DN00IsDENADDataExchangedObject*

```
+ DN00IsDENADDataExchangedObject
+ DN00ScheduleItem (entradas de calendario)
+ DN00AdministrativeNotice (notificaciones administrativas oficiales)
+ DN00AdministrativeOfficialRegisterRecord (registro entrada/salida)
+ DN00AdministrativeServiceProcedureRecord (expediente)
+ DN00DirectDebitPayment (pagos domiciliados)
+ DN00OneOffPayment (pagos singulares en base a liquidación)
```

The process is:

1. Create a [data] retrieve request:

```
DN00DataRetrieveRequest request = DN00DataRetrieveRequestBuilder.retrieveDataType(dataTypeRef)
                                                                    .fromAdmin(adminRef)
                                                                    .forPerson(personRef)
                                                                    .usingClientInstallment(clientInstallmentOid)
                                                                    .lastSuccessfulClientRetrievalAt(...)
                                                                    .build();
```

2. Use the API to retrieve the [data] (*in the sample, [one off payments]-pagos singulares are being retrieved*)

```
DN00DataRetrieveResponse<DN00OneOffPayment> response = _syncAndRetrieveApi.getForDataRetrieveForClientInstallment()
                                                                    .retrieveData(request);
Collection<DN00DataRetrievedFromAdmin<DN00OneOffPayment>> oneOffPayments = response.getData();
```

The returned response is a [model object] of type *DN00DataRetrieveResponse* that contains a collection of *DN00DataRetrievedFromAdmin<D>* (being D the [data type] item model – like *DN00OneOffPayment*-) which in turn contains:

- The [data] itself of type D (like *DN00OneOffPayment*)
- A collection of *DN00DataRetrieveFromAdminProposedScheduleItems* by means of which the [admin] returning the [data] **can also create [schedule items] related with the data**

In the example above of [one off payments] (*pagos singulares*), the [admin] returning the [payments] can also *instruct* DENA-CORE to create multiple [schedule items] for the due-date of each pending [payment]

These [schedule items] will be threatened by DENA-APP as other [schedule items] retrieved from the [admins]

The [proposed schedule items] can be:

- Sent by the [admin] returning the [data]
- Injected at DENA-CORE before returning the [data] to DENA-APP since DENA-CORE has the logic to create these [proposed schedule items] from the [admin] returned [data]